Guion del docente · S2 (240 min)

Las diapositivas tienen notas del ponente (tecla **S**). Este guion añade el ritmo y qué comprobar. Lo que se dice en el aula debe separar lo **comprobado** (pruebas, Linux, GitHub Actions) de lo **sin probar** (Codespace del curso, Docker en equipos propios, Windows).

0:00–0:30 · Revisión de entorno (diap. 1–3)

- Abrir, comprobar audio. Mostrar agenda (diap. 2) en 1 minuto.
- Diap. 3: cada persona ejecuta `conda activate env-ia-mlops`, `cd 04-Proyecto-Base`, `python -m ia_mlops.pipeline` y `python -m pytest -q -m "not lento"`. Esperado: `119 passed, 4 deselected`. Sin `models/modelo_mlflow/` la API no tiene qué servir.
- En el Codespace: `docker version` debe mostrar **Client** y **Server**. Anotar quién no lo consigue y el mensaje.
- Hoy se parte de `develop` limpio, **no** de la rama de la tarea de S1: la variable nueva vuelve a entrar en la tarea de S2.
- Quien falle: pareja. El CI construirá la imagen por esa persona. **No se detiene la sesión.**
- Si pasan los 30 min y hay más de un tercio sin entorno: recortar teoría (diap. 16 y 17), no práctica.

0:30–1:30 · Teoría (diap. 4–17)

Min	Diap.	Mensaje clave	Pregunta a la audiencia
3	4	Un modelo en disco no es un servicio. El modelo es el mismo; cambia lo que lo rodea.	¿Qué le falta a un archivo `.joblib` para que otra persona lo use?
5	5	El contrato: 13 variables con tipo y límites que salen de **cómo se construye FS**, no de lo visto al entrenar.	¿Qué le pasa a un cliente con más facturas que cualquiera de train?
6	6	**Idea central:** el contrato no se escribe dos veces; viaja con el modelo (la firma).	¿Qué ve quien reentrena con una variable nueva y olvida la API?
4	7	Pulsar cada estación. El registro nunca lleva datos del cliente.	¿Qué etapa pone el 422? ¿Cuál el 503?
8	8	**Laboratorio «¿200, 422 o 503?»:** que cada persona prediga el código antes de pulsar. Muestra nueve de los diez casos de archivo (sin `10_frecuencia_cero`) y el servidor sin modelo.	¿Quién se equivocó, quien llama o el servicio?
4	9	Rechazar lo imposible; avisar de lo posible sin respaldo. Mostrar el ticket de 5 000 libras.	¿Qué perdemos si rechazamos ese cliente? ¿Y si callamos?
3	10	Cuatro códigos y quién actúa. El 500 no cuenta nada interno.	¿Qué hace el desarrollador con el `id_peticion`?
3	11	Vivo no es listo. Alternar los dos estados.	¿Por qué reiniciar un servicio sin modelo no lo arregla?

Min	Diap.	Mensaje clave	Pregunta a la audiencia
4	12	Una línea JSON por stdout. Contar el defecto real: sin manejador, con `uvicorn` no salía nada.	¿Por qué una prueba con `TestClient` no lo vio?
5	13	Cuatro caminos, una probabilidad, hasta el último decimal. Mostrar la salida real de `comparar`.	¿Qué cambiaría si el orden de columnas fuera otro?
4	14	Imagen = receta congelada; contenedor = imagen en ejecución. **Ser claros:** el flujo se verificó en GitHub Actions; el Codespace hay que probarlo.	¿Dónde más puede correr esta imagen?
6	15	Pulsar cada bloque del Dockerfile. Tres decisiones: dependencias antes del código, usuario no root, `0.0.0.0`.	¿Por qué `HEALTHCHECK` pregunta a `/salud` y no a `/listo`?
2	16	Lo que no entra importa tanto como lo que entra. La imagen ocupa 826 MB.	¿Qué hay en el contexto de construcción que nunca debería estar?
3	17	El workflow es la plantilla `ci.yml`. No publica nada ni usa secretos.	¿Qué paso del workflow te daría más confianza al hacer un cambio?

1:30–3:45 · Práctica (diap. 18 → `practica/guia-laboratorio.md`)

Ritmo: 10 preparar · 25 arrancar · 30 romper · **10 pausa** · 25 comparar y romper reglas · 25 Docker en Codespaces · 10 CI.

- Recorrer el chat/pantallas a los 15 min de cada bloque.
- **Bloque 2 (romper):** pedir que apunten el código esperado **antes** de cada `curl`. Los casos 08 y 09 (200 con advertencia) son los que más discuten.
- **Bloque 4 (Docker):** es el de más riesgo. Si el Codespace no arranca, esa persona sigue con el bloque 5: el CI construye la imagen. Anotar el mensaje exacto de cada fallo (es información para mejorar el `devcontainer`).
- Quien termine antes: empezar la tarea (reentrenar con una variable nueva y ver el 503 de `/listo`).

3:45–4:00 · Entrega y cierre (diap. 19–21)

- Tarea y rúbrica (diap. 19): dos rutas (A: Online Retail II con la variable de S1; B: caso propio).
- Errores frecuentes (diap. 20) y avance de S3 (diap. 21): la misma imagen va a Cloud Run.
- Recordar: evidencia = pruebas en verde + mensaje 503 del esquema desactualizado + comparación contra el contenedor + imagen (tamaño y usuario) + enlace a una ejecución del workflow.

Si algo falla en directo

Síntoma	Respuesta
`/listo` da 503	Leer el `detalle`: falta el modelo, o el esquema/los metadatos no coinciden con la firma. `python -m ia_mlops.pipeline`.
`COPY failed` en `docker build`	No existe `models/modelo_mlflow/`: entrenar antes.

Síntoma	Respuesta
`docker` no existe en el Codespace	Se abrió sin el `devcontainer`: abrir uno nuevo desde la rama `develop` o seguir con el CI.
`Connection reset by peer` justo tras `docker run`	La API aún arranca: esperar unos segundos y repetir.
Puerto 8080 ocupado	Cerrar la otra API/contenedor o usar `--port 8081`.
`curl` falla en PowerShell	`curl.exe` y comillas dobles: `-d "@ruta"`.
Probabilidad con otros decimales	Esperado entre sistemas operativos; lo que importa es que `comparar` dé diferencias cero.
Una línea `INFO mlflow.agent.hint`	Es de MLflow; se ignora.